

1) Desarrollo orientado a objetos

- Especificación: define qué debe hacer el sistema y sus casos de uso.
- Diseño: decide clases, operaciones/comportamiento y relaciones; suele apoyarse en diagramas.
- Codificación: implementación buscando claridad y eficiencia (memoria y CPU).
- Prueba: operación -> clase -> integración de clases; si falla, volver a diseño/codificación.
- Mantenimiento: corregir errores y ampliar/cambiar funcionalidad; conviene código claro, legible y modular.

2) Ideas esenciales de la POO

- Abstracción: captura las características esenciales y el comportamiento.
- Encapsulamiento: reúne los elementos de una misma entidad; favorece la cohesión.
- Ocultación: la estructura interna queda escondida y se accede mediante métodos/interfaz.
- Herencia: una subclase adquiere atributos y métodos de una superclase.
- Polimorfismo: el mismo mensaje/operación puede provocar comportamientos distintos según el objeto; puede usar enlace dinámico.

3) SOLID

Letra	Principio	Regla para recordar
S	Responsabilidad única	Una clase/módulo: una sola razón para cambiar
O	Abierto/cerrado	Abierto a extensión; cerrado a modificación
L	Sustitución de Liskov	Una subclase debe poder sustituir a la superclase
I	Segregación de interfaces	Mejor interfaces específicas que una general
D	Inversión de dependencia	Depender de abstracciones, no implementaciones

4) Otros principios de diseño

- DRY: evitar duplicaciones lógicas; una funcionalidad debe tener una identidad única.
- IoC: delegar flujos de control/creación y vinculación de objetos para reducir acoplamiento.
- YAGNI: no añadir funcionalidad hasta que sea necesaria.
- KISS: buscar la solución simple y evitar complejidad innecesaria.
- Ley de Demeter: un objeto debe conocer lo mínimo de la estructura interna de sus colaboradores.

5) Clase, objeto y miembros

- Clase: plantilla/tipo que agrupa datos y métodos; existe en tiempo de compilación. Instancia = objeto concreto.
- Objeto: existe en ejecución y tiene estado + comportamiento + identidad.
- Atributos: propiedades que definen el estado. Acceso UML: + público, - privado, # protegido.
- Métodos: manipulan los datos. Constructor crea/inicializa; destructor destruye; getter obtiene; setter modifica.
- Mensaje: solicitud entre objetos; incluye objeto receptor, operación y, en su caso, parámetros. Protocolo = conjunto de mensajes admitidos.

6) Herencia, composición, polimorfismo y sobrecarga

- Herencia = relación "es-un". Simple: una superclase (Java). Múltiple: varias (C++), con posible ambigüedad.
- Clase abstracta: al menos un método sin implementar; no puede instanciarse.
- Composición = relación "tiene-un": crea clases complejas agregando objetos.
- Polimorfismo: misma operación con comportamiento distinto según la clase concreta.
- Sobrecarga: nuevos métodos en la misma clase con distinta signatura. La signatura = nombre + número/tipo de argumentos; NO incluye retorno.
- Sobrescritura: redefinir un método heredado en una clase hija; sí implica herencia.

7) Ventajas e inconvenientes de la OO

- Ventajas: reusabilidad, menos código, extensibilidad/mantenimiento, interoperabilidad, modelado cercano al mundo real, modularidad, bajo acoplamiento y alta cohesión.
- Inconvenientes: no evita malos diseños; requiere formación; mayor curva de aprendizaje; lenguajes OO puros pueden ser menos eficientes; la reutilización exige componentes de calidad.

8) Patrones de diseño: idea y familias

- Patrón = solución reutilizable cuya efectividad se ha comprobado en problemas similares. Los patrones GoF dan vocabulario común y evitan reinventar soluciones.
- Creacionales: aíslan cómo se crean los objetos.
- Estructurales: organizan/conectan objetos para formar estructuras mayores.
- Comportamiento: gestionan acciones, algoritmos y comunicación entre objetos.

9) Patrones creacionales

Patrón	Qué hace
Abstract Factory	Crea familias de objetos relacionados sin clases concretas
Builder	Construye objetos complejos paso a paso
Factory Method	Deja a subclases decidir qué clase instanciar
Prototype	Crea objetos clonando una instancia existente
Singleton	Garantiza una única instancia y acceso global
MVC	Separa datos, interfaz de usuario y lógica de negocio

10) Patrones estructurales

Patrón	Qué hace
Adapter/Wrapper	Convierte una interfaz en otra esperada por el cliente
Bridge	Separa abstracción e implementación
Composite	Jerarquías parte-todo en forma de árbol
Decorator	Añade responsabilidades dinámicamente
Facade	Ofrece una interfaz unificada a un subsistema
Flyweight	Comparte/reduce información redundante entre muchos objetos
Proxy	Representa a otro objeto y controla su acceso

11) Patrones de comportamiento (I)

Patrón	Idea clave
Chain of Responsibility	Cadena de receptores para tratar una petición
Command	Encapsula una petición/operación
Interpreter	Representa una gramática y su intérprete
Iterator	Recorre secuencialmente una colección
Mediator	Centraliza cómo interactúan varios objetos
Memento	Guarda estado para restaurarlo después

12) Patrones de comportamiento (II)

Patrón	Idea clave
Observer	Dependencia 1:N; notifica cambios a dependientes
State	Cambia comportamiento según estado interno
Strategy	Familia de algoritmos intercambiables
Template Method	Esqueleto en superclase; pasos en subclases
Visitor	Añade operaciones sobre una jerarquía sin modificarla

13) GRASP: asignación de responsabilidades

- Experto, Creador, Controlador, Bajo acoplamiento, Alta cohesión, Fabricación pura, Polimorfismo, Indirección y No hable con extraños.
- Objetivo: asignar responsabilidades a clases/objetos de forma que mejore la modularidad y la flexibilidad.

14) Trampas rápidas de patrones

- Factory Method crea un subtipo; Abstract Factory crea familias.
- Prototype clona; Singleton limita a una instancia.
- Adapter cambia interfaz; Facade simplifica un subsistema; Proxy controla acceso.
- Strategy intercambia algoritmos; State cambia conducta por estado; Observer notifica cambios.
- Composite representa parte-todo; Decorator añade responsabilidades sin modificar la clase base.

15) UML: qué es

- UML es un lenguaje gráfico estándar para visualizar, especificar, construir y documentar sistemas.
- Es extensible, escalable y modela desde requisitos hasta artefactos ejecutables usando orientación a objetos.
- No es una metodología: puede usarse con cascada, iterativo, etc.
- El tema indica UML 2.5.1 como versión actual citada y distingue diagramas estructurales y de comportamiento.

18) Diagramas de comportamiento

Diagrama	Objetivo
Actividad	Flujo de actividades/acciones; workflows y algoritmos
Casos de uso	Requisitos funcionales desde el punto de vista del usuario
Máquina de estados	Estados y transiciones provocadas por eventos
Secuencia	Interacción destacando el orden temporal de mensajes
Comunicación	Interacción destacando vínculos entre objetos
Tiempo	Cambio de estado/valor a lo largo del tiempo
Global interacción	Vista global sobre otros diagramas de interacción

16) Diagramas estructurales

Diagrama	Objetivo
Clases	Estructura estática: clases, atributos, métodos y relaciones
Componentes	Organización y dependencia de componentes software
Objetos	Instantánea de instancias en un momento concreto
Estructura compuesta	Estructura interna de una clase y colaboraciones
Despliegue	Particiones físicas/nodos y asignación de software
Paquetes	Organiza el sistema en subsistemas y dependencias

19) Casos de uso e interacción

- Actor: alguien/algo externo que interactúa con el sistema; representa un papel, no necesariamente una persona concreta.
- Caso de uso: secuencia de acciones del sistema que produce un resultado observable y valioso para un usuario.
- Puede especificarse con precondiciones, postcondiciones y escenarios.
- Secuencia: enfatiza orden temporal de mensajes. Comunicación: contiene la misma información básica, pero enfatiza enlaces/colaboraciones y no usa el eje vertical como tiempo.
- Máquina de estados: estado + transición; una transición se dispara por un evento y puede tener condición/acción.

17) Diagrama de clases: relaciones

- Generalización/herencia: relación “es-un”.
- Asociación: vínculo estructural entre clases; puede indicar navegabilidad y multiplicidad.
- Agregación: relación parte-todo débil; los objetos son “parte-de” un todo.
- Composición: agregación fuerte; la vida de las partes está ligada al objeto completo.
- Dependencia: una clase requiere de otra para prestar algún servicio.
- Interfaz: especifica operaciones visibles. Paquete: agrupa elementos del modelo.
- Notación básica de clase: caja con 3 zonas: nombre / atributos / operaciones.

20) Ojo de examen

- Clase = compilación; objeto = ejecución.
- Herencia = es-un; composición = tiene-un.
- Sobrecarga != sobrescritura. La sobrecarga no exige herencia; la sobrescritura sí.
- Retorno NO forma parte de la signatura.
- Abstracta: no se instancia.
- UML no es metodología.
- Despliegue = hardware/nodos; componentes = software; objetos = instantánea; clases = estructura estática.